Unfortunately, the CoffeeScript compiler can't read your mind, and the JavaScript it has produced has a syntax error. Parentheses are important to tell the compiler what to do. You should add parentheses for the function invocations:

```
revenue(salePrice) - cost(salePrice)          <-- Parentheses inside
```

Alternatively, you can put the parentheses on the *outside*:

```
(revenue salePrice) - (cost salePrice)        <-- Parentheses outside
```

Whether you prefer the parentheses inside or outside, you should give ambiguous expressions that contain function invocations parentheses to make them unambiguous.

As a program grows, you don't just keep writing more and more functions. Instead, you often modify and adapt the functions you already have.

6.1.3. Higher-order functions revisited

In chapter 3 you learned the three basic ways that functions are glued together:

- Invocation through variable reference
- As an argument
- As a return value

In listing 6.1 only the first of these was demonstrated. Next, you'll see the other two types of function glue used for generalizing and for partially applying a function.

Agtron isn't selling just PhotomakerExtreme cameras but other types of cameras too. For those other cameras the `overhead`, `costPrice`, and `numberSold` are different from those for the PhotomakerExtreme. The `profit` function in listing 6.1 doesn't work for other types of cameras. How can you generalize it to be used for other types?